A PROJECT REPORT ON

WALLETWISE: AN INTELLIGENT CREDIT CARD REWARDS OPTIMISATION AND LEARNING PLATFORM

SUBMITTED TO THE
VISHWAKARMA INSTITUTE OF TECHNOLOGY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE TERM WORK SUBMISSION OF
ENGINEERING DESIGN AND INNOVATION (EDI) PROJECT

OF THIRD YEAR B.TECH.

SUBMITTED BY

[STUDENT NAME 1]	**Exam No: [EXAM NO.]**
[STUDENT NAME 2]	**Exam No: [EXAM NO.]**
[STUDENT NAME 3]	**Exam No: [EXAM NO.]**



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING - DATA SCIENCE

BRACT'S
VISHWAKARMA INSTITUTE OF TECHNOLOGY, PUNE.

2025 - 2026

CERTIFICATE

This is to certify that the project report entitled

“WALLETWISE: AN INTELLIGENT CREDIT CARD REWARDS OPTIMISATION AND LEARNING PLATFORM”

Submitted by

[STUDENT NAME 1] **Exam No: [EXAM NO.]**
[STUDENT NAME 2] **Exam No: [EXAM NO.]**
[STUDENT NAME 3] **Exam No: [EXAM NO.]**

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of Prof. [Guide Name] and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University.

(Prof. [Guide Name])
Guide
Department of CSE- DS

(Dr. Deepa Abin)
Head,
Department of CSE- DS

Place: Pune
Date :

ACKNOWLEDGEMENT

It gives us great pleasure to present this project report on **WalletWise**. We would like to express our sincere gratitude to our project guide, Prof. [Guide Name], for the continuous guidance, encouragement and constructive criticism that shaped this work at every stage.

We are thankful to Dr. Deepa Abin, Head of the Department of Computer Science and Engineering - Data Science, and to all the faculty members of the department for providing the facilities, the environment and the academic support needed to complete the project.

We also thank the Vishwakarma Institute of Technology, Pune, for giving us the opportunity to work on a real-world, design-oriented problem through the Engineering Design and Innovation course, and the creators of the open-source tools and public resources on which this project depends. Finally, we are grateful to our families and friends for their patience and support.

[STUDENT NAME 1]
[STUDENT NAME 2]
[STUDENT NAME 3]

ABSTRACT

Credit cards in India now return reward points, miles and vouchers that experienced users convert into flights, hotel stays, dining and entertainment. A person who is new to credit cards, however, has no easy way to know which card to swipe at which place, how much value a purchase will return, or how the accumulated points should finally be redeemed. Reward rates differ by category, many benefits depend on issuer portals, and a point may be worth anywhere from a fraction of a rupee to more than a rupee depending on how it is used. As a result, newcomers either leave rewards unclaimed or choose cards at random.

This report presents **WalletWise**, a cross-platform mobile application developed with Expo, React Native and TypeScript that guides a beginner through earning and redeeming credit-card rewards. The application keeps a wallet of the user's cards with fees, benefits, milestones and points balances; recommends the best card for nearby merchants on a map by classifying each place and scoring every card with a transparent rule-based model that returns the reason and the estimated rupee value; provides Aura, a domain-restricted AI assistant that uses a large language model and tool calls to answer questions from the user's own wallet; and offers a Learn hub with topic guides, a points calculator, a quiz and myth-versus-fact cards. The design is based on a review of products such as PointsFly, CardAdvisor and GyFTR.

The recommendation engine was tested on ten merchant and category cases and five classification cases, the code base was checked with the TypeScript compiler and ESLint, and the screens were rendered in a browser build. The tests confirmed the intended recommendations in most cases and exposed one scoring defect and three classification gaps, which are documented with their fixes. WalletWise shows that credit-card optimisation knowledge can be packaged into a clear, premium mobile experience; the planned next stage replaces the hard-coded card data with a verified dataset, adds redemption comparison and a dream-destination tracker, and moves the service keys to a server.

Keywords: *credit card rewards, recommendation engine, React Native, Expo, location-based services, large language model, financial literacy.*

TABLE OF CONTENTS

SR. NO.	TITLE OF CHAPTER	PAGE NO.
	List of Abbreviations	vi
	Nomenclature	vi
	List of Figures	vii
	List of Tables	vii
01	**Introduction**	**1**
1.1	Overview	1
1.2	Motivation	2
1.3	Problem Definition and Objectives	2
1.3.1	Problem Definition	2
1.3.2	Objectives	2
1.4	Project Scope & Limitations	3
1.4.1	Scope	3
1.4.2	Limitations	3
1.5	Methodologies of Problem Solving	4
02	**Literature Survey**	**5**
2.1	Credit-Card Rewards in India	5
2.2	Existing Products and Platforms	5
2.2.1	PointsFly	5
2.2.2	CardAdvisor	5
2.2.3	GyFTR and Voucher Redemption	6
2.3	Enabling Technologies	6
2.4	Comparative Analysis	6
2.5	Research Gap and Positioning	7
03	**Software Requirements Specification**	**8**
3.1	Assumptions and Dependencies	8
3.2	Functional Requirements	8
3.2.1	Wallet Management and Card Detail	9
3.2.2	Location-Based Best Card Recommendation	9
3.2.3	Aura Conversational Assistant	9
3.2.4	Learn Hub	9
3.3	External Interface Requirements	10
3.3.1	User Interfaces	10
3.3.2	Hardware Interfaces	10
3.3.3	Software Interfaces	10
3.3.4	Communication Interfaces	10
3.4	Nonfunctional Requirements	10
3.4.1	Performance Requirements	10
3.4.2	Safety Requirements	10
3.4.3	Security Requirements	11
3.4.4	Software Quality Attributes	11
3.5	System Requirements	11
3.5.1	Database Requirements	11
3.5.2	Software Requirements (Platform Choice)	11
3.5.3	Hardware Requirements	12
3.6	Analysis Models: SDLC Model to be Applied	12
04	**System Design**	**13**
4.1	System Architecture	13
4.2	Mathematical Model	14
4.3	Data Flow Diagrams	15

SR. NO.	TITLE OF CHAPTER	PAGE NO.
4.4	Entity Relationship Diagram	17
4.5	UML Diagrams	18
05	**Project Plan**	**22**
5.1	Project Estimate	22
5.1.1	Reconciled Estimates	22
5.1.2	Project Resources	22
5.2	Risk Management	23
5.2.1	Risk Identification	23
5.2.2	Risk Analysis	23
5.2.3	Overview of Risk Mitigation, Monitoring, Management	23
5.3	Project Schedule	24
5.3.1	Project Task Set	24
5.3.2	Task Network	24
5.3.3	Timeline Chart	25
5.4	Team Organization	25
5.4.1	Team Structure	25
5.4.2	Management Reporting and Communication	25
06	**Project Implementation**	**26**
6.1	Overview of Project Modules	26
6.2	Tools and Technologies Used	27
6.3	Algorithm Details	27
6.3.1	Algorithm 1: Merchant Normalisation and Classification	27
6.3.2	Algorithm 2: Rule-Based Card Scoring	28
6.3.3	Algorithm 3: Nearby Places Pipeline with Stale-Request Guard	28
6.3.4	Algorithm 4: Aura Tool-Calling Loop	28
6.3.5	Algorithm 5: Points Calculator and Expiry Alert	28
07	**Software Testing**	**29**
7.1	Type of Testing	29
7.2	Test Cases & Test Results	29
7.2.1	Recommendation Engine	29
7.2.2	Merchant Classifier	30
7.2.3	Defects Found	30
7.2.4	Static Analysis and Build Verification	31
7.2.5	UI Smoke and Device-Level Tests	31
08	**Results**	**32**
8.1	Outcomes	32
8.2	Screen Shots	32
09	**Conclusions**	**35**
9.1	Conclusions	35
9.2	Future Work	35
9.3	Applications	36
	Appendix A: Feasibility Assessment	**37**
	Appendix B: Details of Paper Publication	**38**
	Appendix C: Plagiarism Report	**38**
	References	**39**

LIST OF ABBREVIATIONS

ABBREVIATION	ILLUSTRATION
AI	Artificial Intelligence
API	Application Programming Interface
COCOMO	Constructive Cost Model
CSV	Comma-Separated Values
DFD	Data Flow Diagram
EDI	Engineering Design and Innovation
ER	Entity Relationship
FR	Functional Requirement
GPS	Global Positioning System
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
KLOC	Thousand Lines of Code
LLM	Large Language Model
LTF	Lifetime Free
MR	Membership Rewards
NFR	Nonfunctional Requirement
OS	Operating System
POI	Point of Interest
REST	Representational State Transfer
SDK	Software Development Kit
SDLC	Software Development Life Cycle
SPPU	Savitribai Phule Pune University
SRS	Software Requirements Specification
UI	User Interface
UML	Unified Modeling Language
UX	User Experience
VIT	Vishwakarma Institute of Technology

NOMENCLATURE

SYMBOL	MEANING
C	Set of cards in the wallet, $C = \{c_1, \ldots, c_n\}$
n	Number of cards in the wallet
m	Normalised merchant name
k	Spending category
a	Purchase amount in rupees (default ₹2,000)
P_c	Catalogue (potential annual) value of card c
B(c, m, k)	Rule bonus added to the score of card c
r(c, m, k)	Effective return rate of the matching rule
V(c, m, k, a)	Estimated rupee value of the purchase, $a \cdot r$
S(c, m, k, a)	Composite score of card c
c*	Recommended (highest-scoring) card
R_E	Earth radius, 6,371 km
φ, λ	Latitude and longitude in radians
d	Haversine great-circle distance in kilometres
E, D, N	COCOMO effort (person-months), time (months), staffing

LIST OF FIGURES

FIGURE	ILLUSTRATION	PAGE NO.
1.1	The reward journey supported by WalletWise	1
3.1	Iterative and incremental development model used for WalletWise	12
4.1	Layered system architecture of WalletWise	13
4.2	Flowchart of the card-scoring algorithm	15
4.3	Data flow diagram, level 0 (context diagram)	16
4.4	Data flow diagram, level 1	16
4.5	Data flow diagram, level 2 (process 2.0, classify and recommend)	17
4.6	Entity relationship diagram	17
4.7	Use case diagram	18
4.8	Sequence diagram: best card near the user	19
4.9	Sequence diagram: Aura answering with a tool call	19
4.10	Class diagram	20
4.11	Component diagram	20
4.12	Deployment diagram	21
4.13	State machine diagram of one Aura exchange	21
5.1	Task network	24
5.2	Timeline (Gantt) chart of the planned fourteen-week schedule	25
6.1	Navigation structure of the application	26
8.1	Schematic of the Best Card (map) screen	33
8.2	Home and Wallet screens	33
8.3	Aura and Learn screens	34
8.4	Card Detail and Topic guide screens	34

LIST OF TABLES

TABLE	ILLUSTRATION	PAGE NO.
2.1	Feature comparison of WalletWise with existing solutions	6
3.1	Summary of functional requirements	8
4.1	Rules and parameters of the scoring model	14
4.2	Data dictionary of the CardData record	18
5.1	Basic COCOMO estimate (organic mode)	22
5.2	Risk analysis (probability and impact rated Low / Medium / High)	23
5.3	Project task set	24
5.4	Team structure and responsibilities	25
6.1	Modules and source files	26
6.2	Technology stack (versions from package.json)	27
7.1	Recommendation engine test cases (spend = ₹2,000)	29
7.2	Merchant normalisation and classification test cases	30
7.3	Defect log	30
7.4	Static analysis and build results	31
7.5	UI smoke tests (web build, 390 × 844 viewport) and device-level tests	31
8.1	Objectives versus outcomes	32

1. INTRODUCTION

1.1 OVERVIEW

Credit cards in India have moved beyond a simple payment instrument. Premium cards now return reward points, air miles, hotel nights and vouchers on everyday spending, and informed users routinely convert these returns into flights, hotel stays, concerts and shopping. The benefit, however, is available only to those who know which card earns the most for which kind of purchase and how the accumulated points should finally be redeemed. A newcomer who has only heard that "credit card points can pay for a vacation" has no practical way of finding out where to swipe which card.

WalletWise is a cross-platform mobile application that closes this knowledge gap. The user records the cards they hold, and the application tells them, for a given merchant, place or spending category, which card to use, how much value the purchase is likely to return, and how that value can later be redeemed. WalletWise combines four ideas in one application: a *card wallet* that stores every card together with its fees, benefits, milestones and points balance; a *location-aware "Best Card" map* that recommends a card for nearby merchants; *Aura*, a conversational assistant for credit-card and rewards questions; and a *Learn hub* that teaches the fundamentals of credit-card rewards to first-time users.

The application is built with Expo and React Native using TypeScript, so one code base runs on Android and iOS, with a premium dark theme, gold accents and a stacked-card wallet. Figure 1.1 summarises the journey that the project supports, from everyday spending to a near-free vacation.

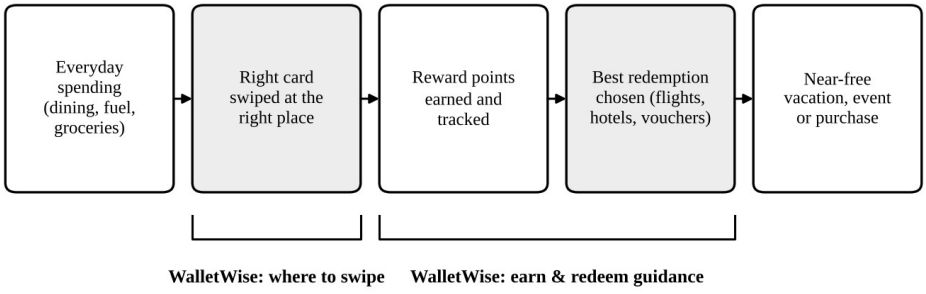


Figure 1.1: The reward journey supported by WalletWise
Shaded stages are the points at which WalletWise intervenes.

1.2 MOTIVATION

Three observations motivated this project. First, reward programmes are complicated. Earn rates differ by merchant category, many cards award accelerated points only through the issuer's own travel portal, points are worth anywhere between a fraction of a rupee and more than a rupee depending on how they are redeemed, and annual fees or spend milestones decide whether a card is profitable at all [1, 2]. A beginner cannot be expected to hold all of this in mind at the point of sale.

Second, the people who benefit most today are enthusiasts who have taught themselves through blogs, videos and forums. Recent Indian products such as PointsFly and CardAdvisor show that there is real demand for tools that turn this know-how into software [3, 1], but they are mostly comparison websites or subscription products for experienced users, and few of them guide a complete beginner through *where to swipe* in the physical world.

Third, redemption is where the value is realised or lost. Points that expire, or are converted at a poor rate, undo the benefit of careful spending. Gift-voucher platforms such as GyFTR allow reward points to be exchanged for vouchers from more than a hundred brands [4], which makes points useful for dining, healthcare, groceries and entertainment as well as travel. WalletWise is motivated by the goal of making the entire loop, earn, track and redeem, understandable to a student or a first-time cardholder.

1.3 PROBLEM DEFINITION AND OBJECTIVES

1.3.1 Problem Definition

To design and implement a mobile application that helps a person who is new to credit cards decide which card to swipe at which place, estimates the reward value of that choice, and explains how accumulated points can be redeemed so that experiences such as travel, dining and entertainment become almost free.

1.3.2 Objectives

- To maintain a digital wallet of the user's credit cards with fees, benefits, milestones, lounge access, forex mark-up and points balances.
- To recommend the best card for a nearby merchant or a spending category using a transparent, rule-based scoring engine.

- To estimate, in rupees, the value a purchase is expected to return so that cards can be compared on a single scale.
- To provide an AI assistant (Aura) that answers rewards questions in natural language and grounds its answers in the user's own wallet through tool calls.
- To teach credit-card concepts through a Learn hub containing topic guides, an interactive calculator, a quiz and myth-versus-fact cards.
- To warn the user when reward points are about to expire.
- To deliver a polished, responsive interface that follows mobile-first design and works on both Android and iOS.

1.4 PROJECT SCOPE & LIMITATIONS

1.4.1 Scope

The delivered system is a client-side mobile application with five main screens (Home, Best Card, Aura, Learn and Wallet), two supporting screens (Card Detail and Topic guide) and a short launch animation. The wallet is pre-loaded with five Indian premium cards: HDFC Infinia, SBI BPCL Octane, American Express Platinum Travel, HDFC Regalia Gold and HDFC Marriott Bonvoy. The Best Card screen works for any location that the Mapbox geocoding service can resolve, and the Learn hub contains a ranked guide to ten travel cards.

1.4.2 Limitations

- Card data and recommendation rules are hard-coded in the application. They are not yet loaded from a verified dataset, so rates may differ from the issuers' current terms.
- The wallet is stored in memory only. Cards cannot yet be added, edited or deleted by the user, and nothing is persisted between launches.
- Home-screen balances, activity and the rewards-optimiser demonstration are illustrative values and are not computed from real transactions.
- The recommendation engine considers a single purchase at a time. It does not yet model monthly caps, spend milestones or annual-fee break-even across a portfolio.

- The Groq and Mapbox keys are read from public environment variables and therefore ship inside the application bundle. A production release must route these calls through a back-end proxy.

1.5 METHODOLOGIES OF PROBLEM SOLVING

The problem was approached in four steps. *Domain study*: reward structures, point valuation and redemption options of the five cards were studied, together with the public descriptions of existing products [3, 1, 4]. *Decomposition*: the problem was divided into independent concerns, namely data (card store), reasoning (merchant classification and scoring), perception (location and place search), conversation (Aura) and education (Learn), so that each could be built and tested separately. *Incremental construction*: the wallet and card detail were built first, then the map and the engine, then Aura and the Learn hub, as described in Section 3.6. *Evaluation*: the recommendation engine was exercised with representative merchants, the code base was checked with the TypeScript compiler and ESLint, and the screens were rendered in a browser build, as reported in Chapter 7.

For the recommendation itself, a transparent rule-based model was preferred over a learned one: reward rules are published, few and slow to change, so explicit rules can be audited and explained to a beginner ("7.25% value back at BPCL"). The conversational layer uses a large language model because free-form questions cannot be anticipated; its answers are constrained by a strict system prompt and by tools that return data from the application.

2. LITERATURE SURVEY

2.1 CREDIT-CARD REWARDS IN INDIA

Indian credit cards reward spending in bank points, co-branded programme points such as Marriott Bonvoy, and cashback. The value of a point is not fixed: CardAdvisor reports a range of roughly ₹0.20 to over ₹1.00 depending on issuer and redemption [1], and defines effective return as earn rate multiplied by point value, divided by spend [2]. A point used for a statement credit can be worth several times less than one transferred to an airline or hotel partner or exchanged for a voucher. Milestones, lounge visits, fuel-surcharge waivers, fee waivers and issuer travel portals add further layers, so choosing a card is a decision problem rather than a lookup.

2.2 EXISTING PRODUCTS AND PLATFORMS

2.2.1 PointsFly

PointsFly is a recent AI-native application for Indian cardholders. According to public descriptions, it centres on an agent named AIRA that selects cards, tracks points and helps book flights and hotels with points; it offers a wallet of cards and balances, a best-card suggestion for a store or purchase, award search that shows cash and points prices with a value per point, a redemption valuation that converts every option into rupees, a "Dream Destination" goal that everyday spending works towards, and nearby store alerts [3, 5]. It is offered on a subscription basis after a trial period [5]. PointsFly is the most important reference for WalletWise because it demonstrates how reward optimisation can be packaged as a consumer application.

2.2.2 CardAdvisor

CardAdvisor is a recommendation and information website. It ranks cards by estimated annual rupee value after fees using a user's spending profile, supports wallet gap analysis and side-by-side comparison, and publishes an open dataset of 279 cards under a CC BY 4.0 licence whose reward rates deliberately exclude issuer-portal accelerators [1, 6]. Its structured data and its effective-return formula [2] are directly relevant to the next stage of WalletWise, in which the hard-coded card list will be replaced by a dataset-driven catalogue.

2.2.3 GyFTR and Voucher Redemption

GyFTR is an Indian gift-voucher platform on which cardholders can pay for vouchers from more than a hundred brands with reward points, or combine points with a card payment when the balance is short; the number of points required depends on the brand, the face value and the card variant [4]. Bank programmes also run promotions through the platform; for example, Axis Bank's terms describe accelerated EDGE reward points on purchases made through a dedicated GyFTR page, with exclusions, and note that purchased vouchers cannot be cancelled or exchanged [7]. Voucher platforms extend redemption beyond travel to groceries, healthcare, dining and entertainment. Bank portals such as HDFC SmartBuy, in contrast, are authoritative for one issuer but cannot compare competing banks' cards.

2.3 ENABLING TECHNOLOGIES

Location services. The Mapbox Geocoding API converts a text query such as "restaurant" into ranked places with coordinates and an optional proximity bias [8]; distances are computed with the haversine formula [9] using device position from the platform location API [10]. *Conversational AI.* Language models accept tool (function) definitions and answer with text or a structured tool request; the application executes the tool and returns the result so the model can compose a grounded answer. WalletWise uses this pattern through Groq's hosted endpoint [11] with the open-weight gpt-oss-120b model [12]. *Cross-platform development.* React Native renders native views from JavaScript [13]; Expo provides the tool chain and Expo Router file-based routing [14, 15]; Zustand supplies global state [16], Reanimated native-thread animation [17] and react-native-maps the map [18].

2.4 COMPARATIVE ANALYSIS

Table 2.1 compares WalletWise with the products reviewed above. The comparison is based on publicly described features and on the delivered WalletWise application; "planned" marks features that are part of the roadmap in Section 9.2.

Table 2.1: Feature comparison of WalletWise with existing solutions

Feature	PointsFly	CardAdvisor	Bank portal	WalletWise
Multi-card wallet	Yes	Yes (gap analysis)	Own cards only	Yes

Feature	PointsFly	CardAdvisor	Bank portal	WalletWise
Best card at a place	Yes	No	No	Yes (map-based)
Estimated ₹ value of a swipe	Yes	Annual estimate	Partial	Yes (per swipe)
AI assistant	Yes (AIRA)	No	No	Yes (Aura)
Redemption comparison	Yes	Valuations	Own options	Planned
Voucher redemption guidance	Partial	No	Partial	Planned
Travel goal tracking	Yes	No	No	Planned
Beginner education hub	Limited	Articles	Limited	Yes (Learn)
Open card dataset	No	Yes (279 cards)	No	Planned (import)

2.5 RESEARCH GAP AND POSITIONING

The survey reveals a gap between sophisticated tools for experienced users and the needs of a beginner. Existing comparison sites do not operate at the point of sale, and the applications that do (such as PointsFly) are subscription products. WalletWise positions itself as an educational, student-friendly application that unites three things in one interface: where to swipe (the Best Card map), why (Aura's explanations and the Learn hub) and what to do with the points (redemption guidance). It adopts proven ideas from the reviewed products, namely the wallet, best-card suggestion, rupee valuation and goal-based motivation, while concentrating on clarity for first-time cardholders.

3. SOFTWARE REQUIREMENTS SPECIFICATION

This chapter follows the structure recommended by ISO/IEC/IEEE 29148 [19] to state what the system must do and the conditions under which it operates.

3.1 ASSUMPTIONS AND DEPENDENCIES

- The user owns a smartphone running Android or iOS and has an internet connection while using the Best Card and Aura features.
- The user grants foreground location permission for the Best Card feature. Without it, the user can still search for a place manually.
- Mapbox and Groq services are available and valid access keys are supplied through environment variables at build time.
- Card terms stored in the application reflect the issuers' published terms at the time of data entry; they may change without notice.
- Recommendations are informational and do not constitute financial advice.

3.2 FUNCTIONAL REQUIREMENTS

Table 3.1 summarises the functional requirements, whose identifiers are used in the test plan.

Table 3.1: Summary of functional requirements

ID	Feature	Requirement
FR-1	Intro animation	Play a branded video on launch, hide the native splash screen on first frame, and continue to the app on completion, error or a 15-second timeout.
FR-2	Home dashboard	Show a greeting, financial snapshot, Aura entry point, rewards optimiser, quick actions, recent activity and reward summary.
FR-3	Wallet	Display the cards as an expandable, stacked carousel with portfolio value and a points-expiry banner.
FR-4	Card detail	For a selected card, show points balance, estimated value range and tabs for Earn, Transfers, Redeem, Benefits and Offers.
FR-5	Best Card map	Locate the user, list nearby merchants by category, classify each merchant and show the best card with estimated value and reason.
FR-6	Place search	Provide debounced autocomplete search and re-centre the map and recommendations on a chosen place.

ID	Feature	Requirement
FR-7	Recommendation	Rank all wallet cards for a merchant and category and return score, estimated rupee value and a human-readable reason.
FR-8	Aura assistant	Answer credit-card questions in a chat interface, call tools to read the wallet and compute the best card, and refuse unrelated topics.
FR-9	Learn hub	Provide a featured video, topic guides, card-specific lessons, points calculator, quiz, myth-versus-fact cards and an academy path.
FR-10	Topic guide	Show a ranked list of cards for a topic with tier, fees, forex, reward range and a link to the issuer's page.

3.2.1 Wallet Management and Card Detail

The wallet reads its cards from a single store. Tapping a card opens the detail screen with the card art as a shared element, the points balance in the card's own currency and a low-to-high estimated value, followed by five tabs: Earn (base return, categories, fees), Transfers (forex mark-up, partners), Redeem (best value and a plain-language translation), Benefits (lounge access, milestones) and Offers.

3.2.2 Location-Based Best Card Recommendation

On opening the Best Card tab the application requests location permission and watches the device position. For the active category it queries the geocoding service, enriches every place with a normalised name, category, distance and ranked recommendation, and lists the places by distance in a bottom sheet. The user can change the category chip, search another area, or tap a recommendation to open the card.

3.2.3 Aura Conversational Assistant

The conversation and tool definitions are sent to the language model. If it requests a tool, the application runs it against the card store and asks again until a plain-text answer results. The system prompt restricts Aura to credit-card and finance topics and forbids invented rates, fees or eligibility.

3.2.4 Learn Hub

The Learn tab offers a featured video, eight topic pills linking to topic guides, a carousel of the user's cards, a points calculator, a four-question quiz, a myth-versus-fact flip card and a four-stage academy path.

3.3 EXTERNAL INTERFACE REQUIREMENTS

3.3.1 User Interfaces

The interface uses a dark theme (background #0A0A0A, surface #141414) with a gold gradient accent (#F5D06F to #C9962E) and light text (#F5F5F5). Navigation is a five-tab bar with a raised central Aura button; interactions use spring animations, press-scale feedback and a draggable bottom sheet with three snap positions.

3.3.2 Hardware Interfaces

The application uses the device GPS receiver, the touch screen and the network interface; no other hardware is required.

3.3.3 Software Interfaces

The application consumes the Mapbox Geocoding API (v5) and raster tiles, the Groq chat-completion API, Expo location, video, image and web-browser modules, and YouTube thumbnails.

3.3.4 Communication Interfaces

All external communication uses HTTPS with JSON payloads; Groq requests carry a bearer API key and Mapbox requests an access token.

3.4 NONFUNCTIONAL REQUIREMENTS

3.4.1 Performance Requirements

Animations run on the native thread through Reanimated, place search is debounced by 400 ms, and recommendations for twelve places are computed in milliseconds because the engine is a linear pass over a short list.

3.4.2 Safety Requirements

Because the application discusses money, estimates must not be presented as guarantees: Aura is instructed to say "I don't have verified information for that" when data is missing, and values are labelled as estimates.

3.4.3 Security Requirements

Location is requested only in the foreground and is not stored. API keys are bundled with the client in this prototype, which is recorded as a risk with a server proxy as the planned mitigation. No card numbers, CVV codes or credentials are collected; only the last four digits are kept for display.

3.4.4 Software Quality Attributes

- **Usability:** plain-language reasons ("7.25% value back on BPCL fuel"), consistent navigation and a beginner-oriented Learn hub.
- **Maintainability:** typed models, a single card store, separate utility modules for classification, scoring and geometry, and file-based routes.
- **Portability:** one TypeScript code base for Android, iOS and (partially) web through React Native and Expo.
- **Reliability:** stale-request guards on asynchronous searches, a launch fallback timeout, and an apology message when a network call to the language model fails.
- **Extensibility:** the engine and card model can be driven by an external dataset without changing the screens.

3.5 SYSTEM REQUIREMENTS

3.5.1 Database Requirements

The prototype keeps its data in an in-memory Zustand store holding an array of card records (fields in Section 4.4). AsyncStorage is installed so that the wallet can be persisted locally in the next iteration; no server database is required yet.

3.5.2 Software Requirements (Platform Choice)

Development runs on Windows, macOS or Linux with Node.js 18 or newer (npm or Bun), Expo SDK 57, React Native 0.86, React 19.2 and TypeScript 6 with ESLint. Testing uses Expo Go or a development build on Android/iOS and Chromium for the web preview, and the services require a Mapbox access token and a Groq API key. Source control is Git and GitHub.

3.5.3 Hardware Requirements

Development needs a dual-core machine with 8 GB RAM, 5 GB of free disk space and internet access. The target device is an Android or iOS phone with GPS and network connectivity; the application is locked to portrait orientation.

3.6 ANALYSIS MODELS: SDLC MODEL TO BE APPLIED

The project follows an *iterative and incremental* model [20, 21]. Requirements were fixed at a high level, and the product was grown in three increments, each passing through requirement analysis, design, implementation, testing and review before the next began (Figure 3.1). This suits a student project in which the interface is refined by repeated demonstration and in which the riskiest parts, the recommendation rules and the language-model integration, benefit from early feedback. A strict waterfall model was rejected because the look and behaviour of the screens could not be specified fully in advance.

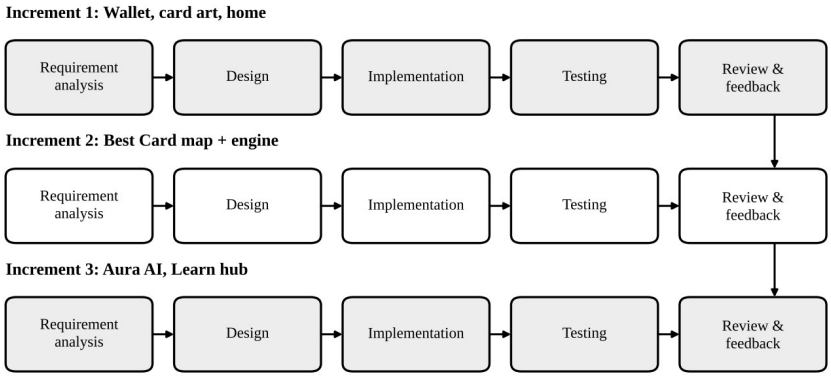


Figure 3.1: Iterative and incremental development model used for WalletWise

4. SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

WalletWise is organised in five layers (Figure 4.1). The *presentation layer* consists of Expo Router screens, one file per route under the `src/app` directory. The *UI component layer* holds reusable pieces such as the card artwork components, the launch video and the animated pressable and reveal wrappers, together with the theme tokens. The *business logic layer* contains the `useBestCardLogic` hook, the recommendation engine, the merchant classifier, the haversine utility and the Aura tool executor. The *state layer* is a single Zustand store that is the source of truth for the wallet. Beneath these, the application talks over HTTPS to Mapbox and Groq and reads the device position through the operating system.

This separation keeps screens thin, and because the engine and classifier are pure functions without React dependencies they could be tested outside the application, as in Chapter 7.

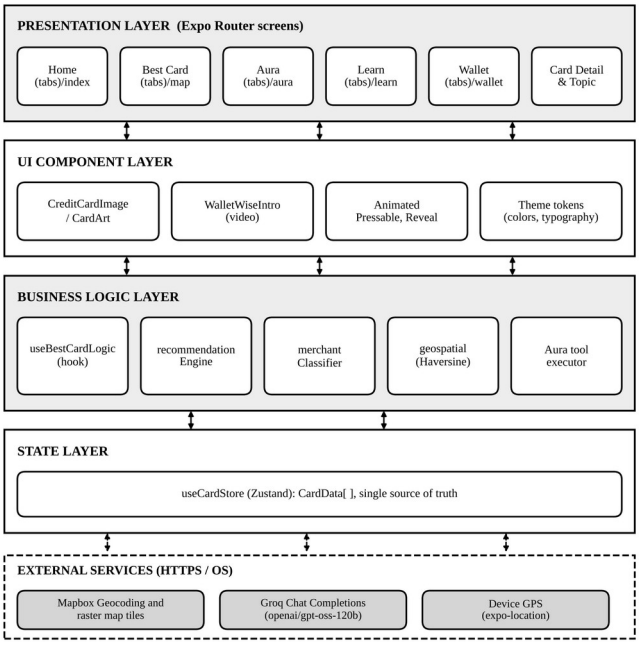


Figure 4.1: Layered system architecture of WalletWise

Solid arrows: in-process calls. Dashed arrows: network or operating-system calls.

4.2 MATHEMATICAL MODEL

Let $C = \{c_1, \ldots, c_n\}$ be the set of cards in the wallet, m a normalised merchant name, k a spending category and a the purchase amount in rupees (default ₹2,000 when unspecified). Each card carries a catalogue value P_c, its estimated potential annual value. The engine assigns every card the score

$$S(c, m, k, a) = P_c / 10^4 + B(c, m, k) + V(c, m, k, a), \qquad V(c, m, k, a) = a \cdot r(c, m, k)$$

where B is a bonus awarded when a rule matches (Table 4.1) and r is the rule's effective return rate. Rules are evaluated as an ordered chain: exact-merchant rules first, then category rules; if no rule yields a positive value, a fallback base rate is used. The recommended card is

$$c^* = \text{argmax over } c \text{ in } C \text{ of } S(c, m, k, a)$$

and the displayed rupee benefit is $V(c^*, m, k, a)$. Because the bonus B dominates the catalogue term (at most about 12 for the cards in the wallet), a matching rule decides the ranking, while P_c only breaks ties among cards that match no rule. The distance shown beside each place is the haversine great-circle distance [9]

$$d = 2 R_E \cdot \arcsin\left(\sqrt{ \sin^2(\Delta\varphi/2) + \cos \varphi_1 \cdot \cos \varphi_2 \cdot \sin^2(\Delta\lambda/2) } \right)$$

with Earth radius $R_E = 6{,}371$ km, latitudes φ and longitudes λ in radians. Scoring one purchase costs $O(n \cdot |R|)$ time for n cards and a constant rule set R, and ranking adds $O(n \log n)$; both are negligible for wallets of fewer than a few dozen cards. Figure 4.2 shows the procedure.

Table 4.1: Rules and parameters of the scoring model

Condition (m or k)	Card matched	Bonus B	Rate r	Reason shown to the user
Merchant BPCL	BPCL Octane	100	7.25%	7.25% value back on BPCL fuel
Merchant Marriott	Marriott Bonvoy	100	15%	Accelerated Bonvoy points
Merchant Taj	Platinum Travel	80	10%	Excellent for Taj milestone redemptions
Category fuel	BPCL Octane	50	2.5%	Great fuel card
Category fuel	Any card with a fuel-surcharge waiver	20	1%	1% fuel surcharge waiver

Condition (m or k)	Card matched	Bonus B	Rate r	Reason shown to the user
Category dining	Infinia	60	33%	Up to 33.3% via SmartBuy vouchers
Category dining	Diners Club	50	10%	10X points on dining partners
Travel / flight / hotel	Atlas	70	10%	Accelerated EDGE miles
Travel / flight / hotel	Infinia or Diners	65	16%	16.6%–33.3% return via SmartBuy
Shopping / electronics	Infinia	40	16%	16.6% via SmartBuy
Shopping / electronics	Regalia	30	5%	5% via SmartBuy
No rule matched	Infinia, Diners	0	3.3%	3.3% base reward rate
No rule matched	Atlas	0	2%	2% base EDGE miles
No rule matched	All others	0	1.5%	Standard reward points

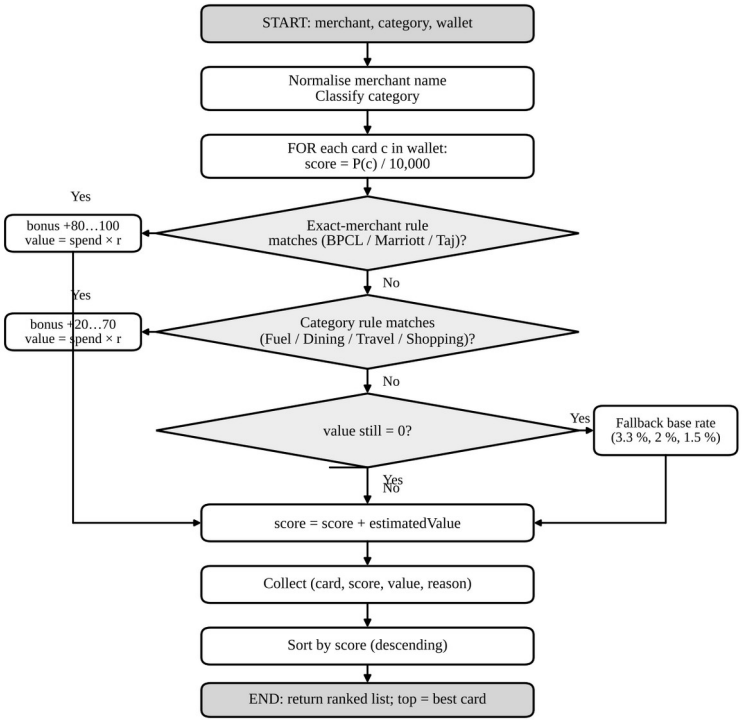


Figure 4.2: Flowchart of the card-scoring algorithm

4.3 DATA FLOW DIAGRAMS

The context diagram (Figure 4.3) shows WalletWise as one process exchanging data with the user and three external parties: Mapbox for places and tiles, the Groq language model for chat, and the device GPS. Level 1 (Figure 4.4) decomposes the system into five

processes that share one data store, the card store: locating and discovering places, classifying and recommending, managing the wallet and card detail, the Aura chat, and the Learn hub. Level 2 (Figure 4.5) expands process 2.0 into name normalisation, category classification, card scoring and ranking.

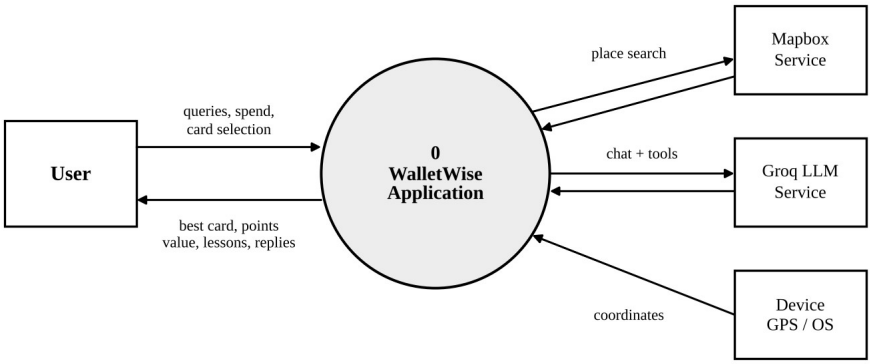


Figure 4.3: Data flow diagram, level 0 (context diagram)

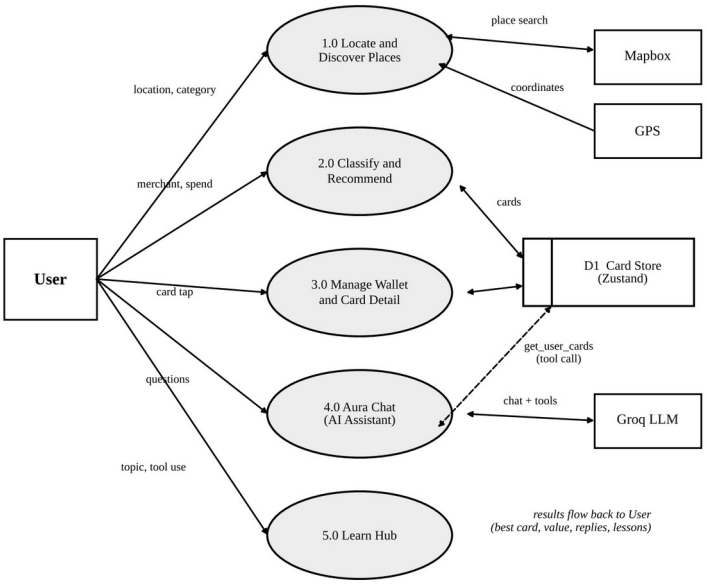


Figure 4.4: Data flow diagram, level 1

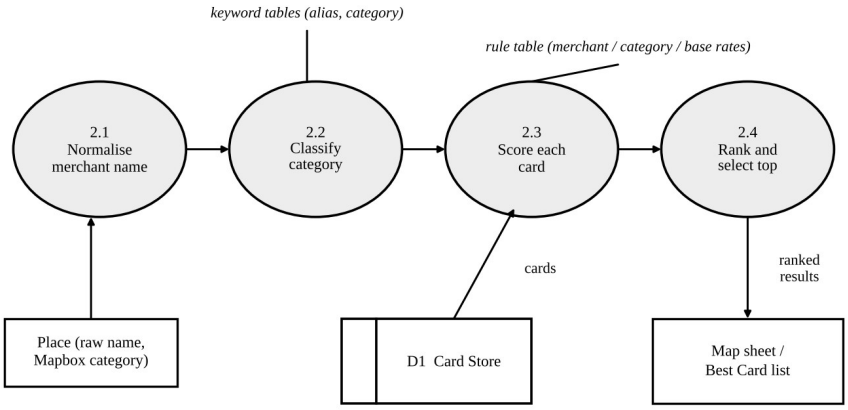


Figure 4.5: Data flow diagram, level 2 (process 2.0, classify and recommend)

4.4 ENTITY RELATIONSHIP DIAGRAM

Because the prototype has no server database, the model describes in-memory records. CARD owns one INTELLIGENCE record and several list-valued attributes; a PLACE returned by Mapbox is scored against every CARD, giving RECOMMENDATION records (a many-to-many relationship resolved by card id and place id); TOPIC CARD records belong to the Learn hub; CHAT MESSAGE records hold the Aura conversation. Table 4.2 is the data dictionary of the card record.

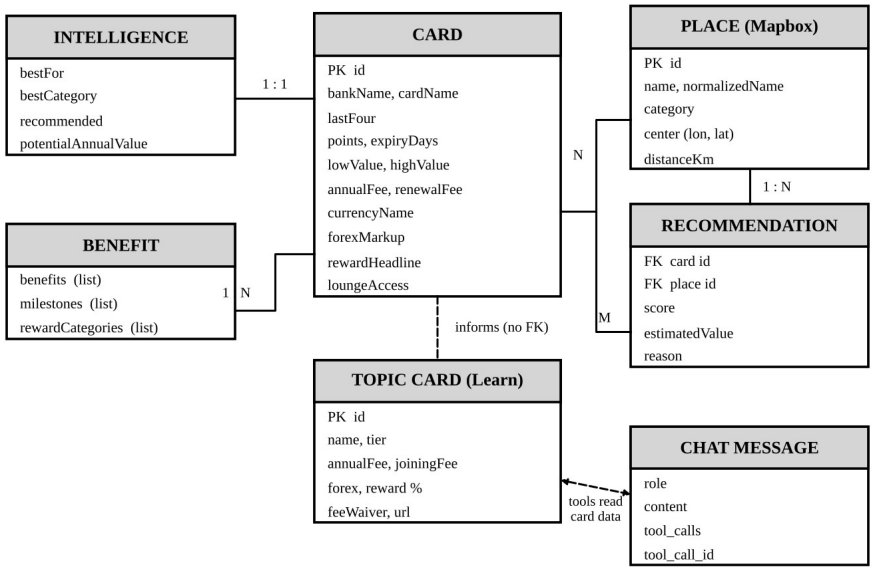


Figure 4.6: Entity relationship diagram

Table 4.2: Data dictionary of the CardData record

Field	Type	Meaning
id, bankName, cardName, lastFour	string	Route identifier, issuer, product name and last four digits
points, expiryDays?	number	Points balance and optional days left before expiry
lowValue, highValue	number	Estimated rupee value range of the balance
annualFee, renewalFee, forexMarkup	string	Fee text with waiver conditions; foreign-currency mark-up
currencyName, rewardHeadline, translatorExample	string	Reward currency, headline earn rate, plain-language redemption example
benefits, milestones, rewardCategories	string[ ]	Lists shown in the Benefits and Earn tabs
loungeAccess	string	Airport lounge entitlement
intelligence	object	bestFor, bestCategory, recommended (Yes/No), potentialAnnualValue (P_c)

4.5 UML DIAGRAMS

The Unified Modeling Language [22] is used to describe behaviour and structure. The use case diagram (Figure 4.7) lists what the user can do and which external actors participate. Figures 4.8 and 4.9 are sequence diagrams of the two most important interactions, finding the best card near the user and asking Aura a question that needs a tool call.

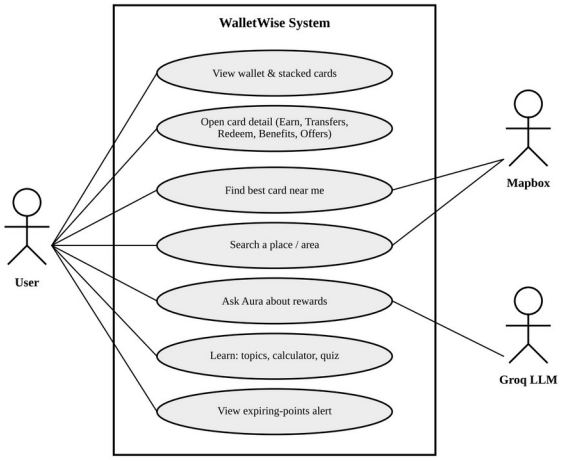


Figure 4.7: Use case diagram

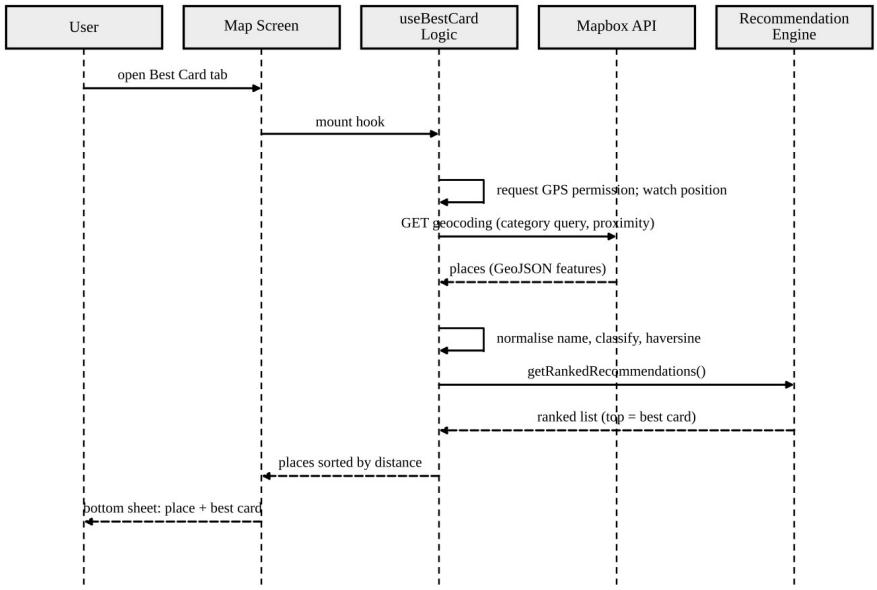


Figure 4.8: Sequence diagram: best card near the user

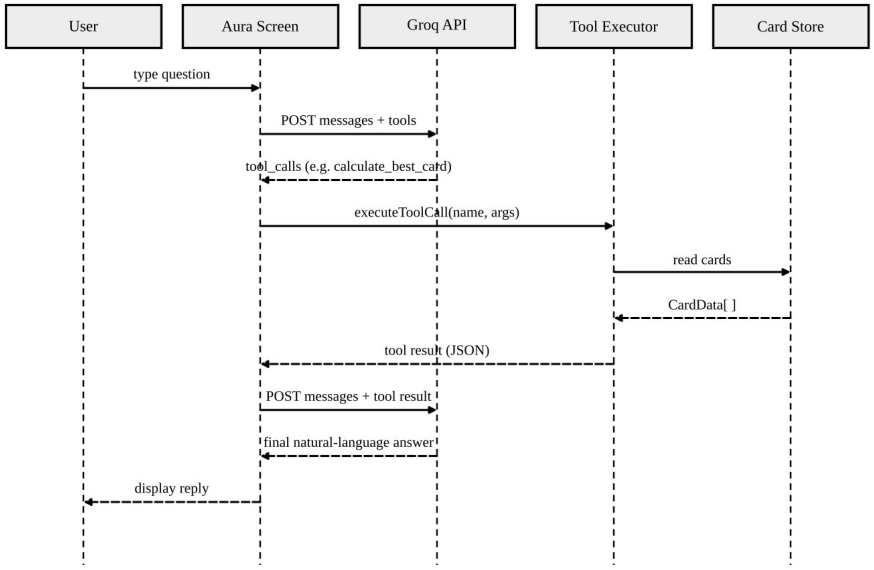


Figure 4.9: Sequence diagram: Aura answering with a tool call

The class diagram (Figure 4.10) shows the principal types; the component diagram (Figure 4.11) maps them to libraries; the deployment diagram (Figure 4.12) shows the device and the Mapbox and Groq clouds; and the state machine (Figure 4.13) captures one Aura message, including the loop that repeats while the model keeps requesting tools.

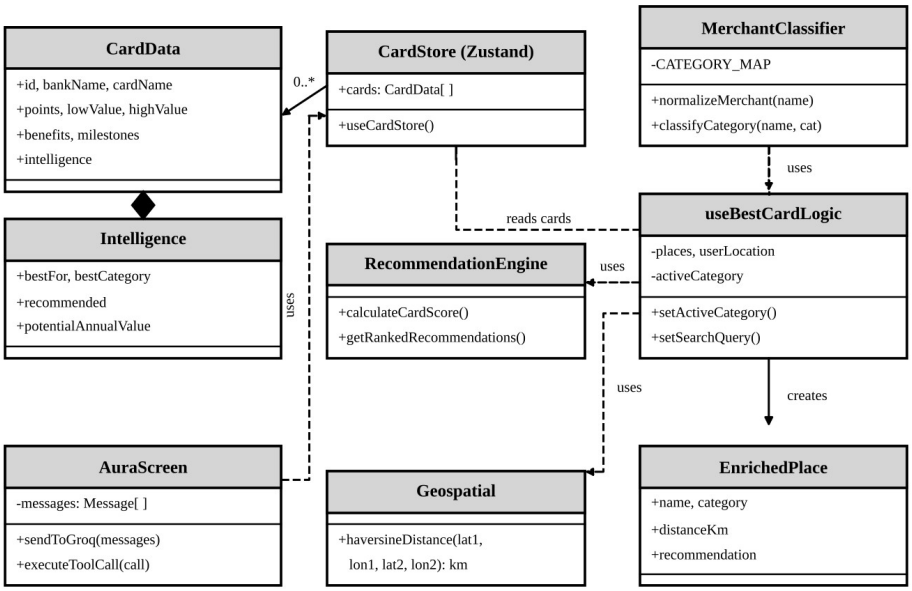


Figure 4.10: Class diagram

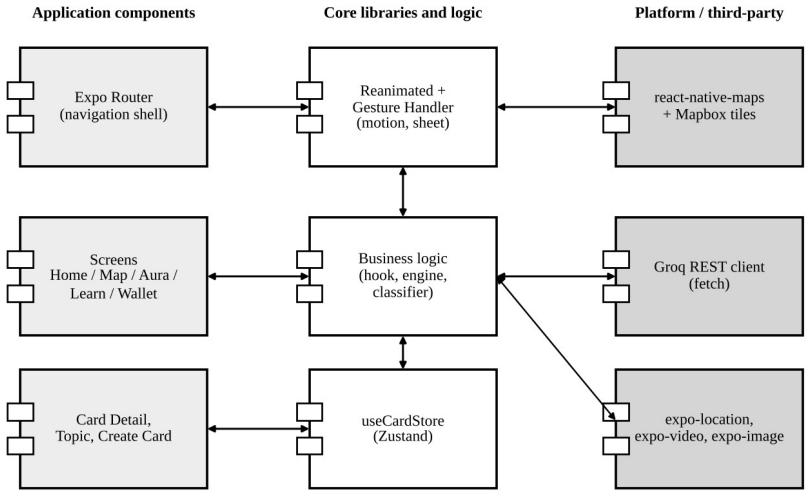


Figure 4.11: Component diagram

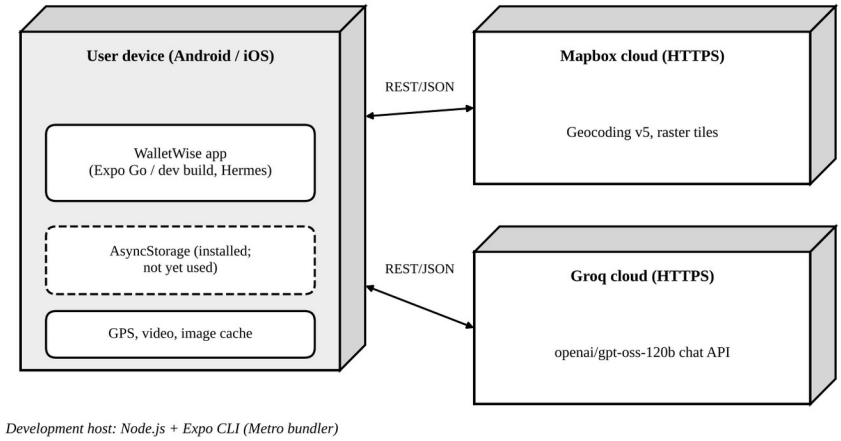


Figure 4.12: Deployment diagram

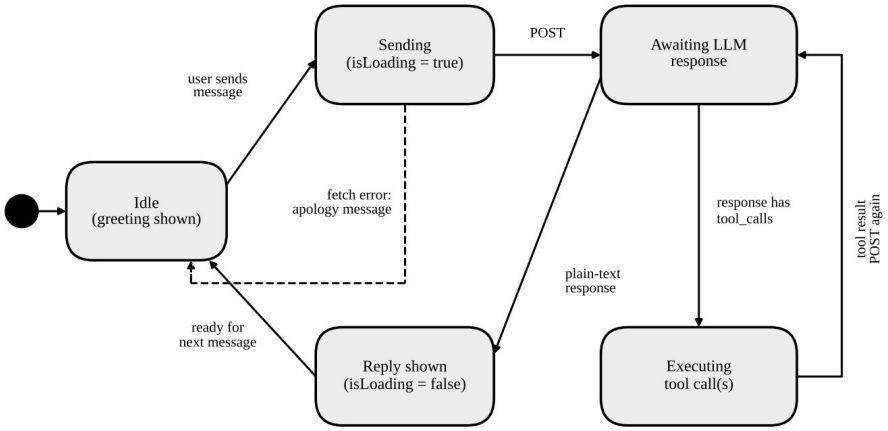


Figure 4.13: State machine diagram of one Aura exchange

5. PROJECT PLAN

5.1 PROJECT ESTIMATE

5.1.1 Reconciled Estimates

Effort was estimated with the Basic COCOMO model in organic mode [23], effort E = 2.4 (KLOC)^1.05 person-months and development time D = 2.5 (E)^0.38 months. The delivered source tree under `src/` contains 6,234 lines of TypeScript, TSX and CSS, that is about 6.2 KLOC. Table 5.1 shows the result.

Table 5.1: Basic COCOMO estimate (organic mode)

Quantity	Formula	Value
Size	Measured from src/	6.2 KLOC
Effort	E = 2.4 × 6.2^1.05	≈ 16.3 person-months
Development time	D = 2.5 × E^0.38	≈ 7.2 months
Average staffing	N = E / D	≈ 2.3 persons

COCOMO assumes code written from scratch, whereas WalletWise stands on Expo, React Native and Reanimated and many of its lines are declarative styles, so the model over-estimates. The estimate was reconciled with a fourteen-week term (about 3.2 months) and a team of two to three students; the shorter schedule was met by reuse and by limiting scope to the three increments of Section 3.6. The COCOMO figure is kept as an upper bound.

5.1.2 Project Resources

The project uses development laptops and Android/iOS phones; Node.js, Expo CLI, Visual Studio Code, Git/GitHub, Chromium and Expo Go; the libraries of Table 6.2; the Mapbox and Groq cloud services; and issuer websites for card terms, with the CardAdvisor open dataset planned.

5.2 RISK MANAGEMENT

5.2.1 Risk Identification

Risks were identified from the architecture and from the third-party dependencies: technical (API availability, key exposure, model errors), data (outdated card terms) and project (schedule, scope) risks.

5.2.2 Risk Analysis

Table 5.2: Risk analysis (probability and impact rated Low / Medium / High)

#	Risk	Prob.	Impact	Mitigation
R1	API keys extracted from the app bundle	High	High	Move calls behind a server proxy; rotate keys; restrict token scopes and quotas
R2	Mapbox or Groq outage or rate limit	Medium	Medium	Catch errors, show a friendly message; cache results; fall back to manual category browsing
R3	Card terms change or are entered incorrectly	High	High	Import a maintained dataset; show "estimate" labels; record the verification date
R4	Language model gives wrong or off-topic answers	Medium	High	Strict system prompt, low temperature (0.2), tools for data, refusal rule, disclaimer
R5	Sparse or mislabelled places from geocoding	Medium	Medium	Keyword classifier with category fallback to the selected chip; add aliases
R6	Location permission denied	Medium	Low	Allow manual place search; stop the loading state gracefully
R7	Scope creep from reference products	Medium	Medium	Fix increments; record extras in the roadmap
R8	Native-only modules unavailable on web	High	Low	Test the map on devices; use the web build only for non-map screens

5.2.3 Overview of Risk Mitigation, Monitoring, Management

Each risk has an owner and is reviewed at the end of every increment. The highest-priority items, R1 and R3, are carried into the roadmap of Section 9.2.

5.3 PROJECT SCHEDULE

5.3.1 Project Task Set

Table 5.3: Project task set

ID	Task	Main deliverable	Weeks
A	Requirement study and literature survey	Feature list; competitor comparison	1–2
B	UI/UX design and theme	Colour tokens, typography, tab layout	2–4
C	Wallet, card art and card detail	Stacked wallet, five-tab card detail	3–6
D	Map, geocoding and recommendation engine	Best Card screen, classifier, scorer	5–9
E	Aura assistant and tools	Chat UI, Groq integration, tool executor	8–11
F	Learn hub and topic guides	Calculator, quiz, topic list	9–12
G	Testing, static analysis, documentation	Test results, this report	11–14

5.3.2 Task Network

Figure 5.1 shows the dependencies among the tasks. Task D (the map and engine) depends on both the design (B) and the data model created in C, and the Learn hub (F) can only be finalised when the engine and Aura exist. The critical path is A, B, D, F, G.

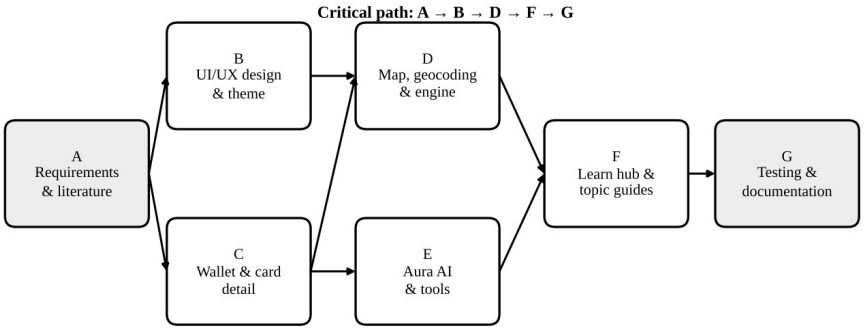


Figure 5.1: Task network

5.3.3 Timeline Chart

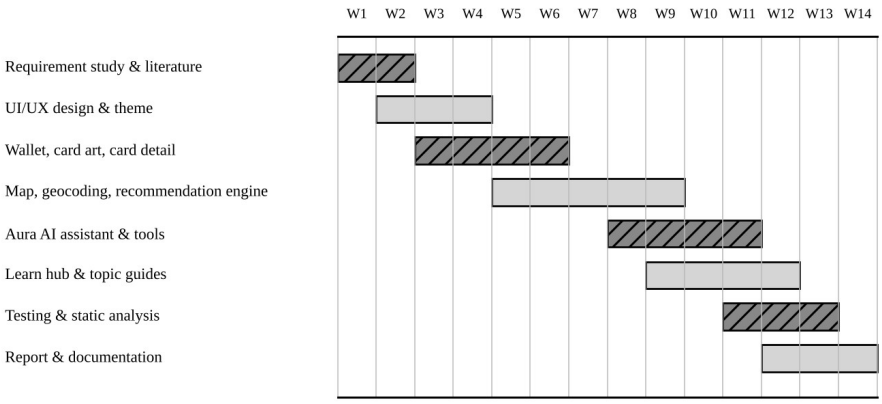


Figure 5.2: Timeline (Gantt) chart of the planned fourteen-week schedule

5.4 TEAM ORGANIZATION

5.4.1 Team Structure

The project is carried out by a small student team working under an internal guide. Responsibilities are shared by area, as in Table 5.4, while every member reviews the code of the others.

Table 5.4: Team structure and responsibilities

Role	Member	Primary responsibility
Project guide	Prof. [Guide Name]	Technical direction, review of reports
Team member 1	[Student Name 1]	Mobile UI, wallet and card detail
Team member 2	[Student Name 2]	Recommendation engine and map
Team member 3	[Student Name 3]	Aura assistant, Learn hub, documentation

5.4.2 Management Reporting and Communication

The team met the guide weekly to demonstrate a working build, note defects and agree the next tasks. Source code is held in a GitHub repository, work is committed in small steps, and the README documents setup.

6. PROJECT IMPLEMENTATION

6.1 OVERVIEW OF PROJECT MODULES

The code base is organised by responsibility (Table 6.1). Routes are files; the tab group `(tabs)` holds the five main screens, and `card-detail`, `topic/[id]` and `create-card` are pushed on top of the tabs (Figure 6.1).

Table 6.1: Modules and source files

Module	Source file(s)	Responsibility
Root layout and intro	app/_layout.tsx, WalletWiseIntro.tsx	Splash handling, intro video, navigation stack
Home	app/(tabs)/index.tsx	Dashboard, Aura entry point, rewards optimiser
Best Card	app/(tabs)/map.tsx, hooks/useBestCardLogic.ts	Map, bottom sheet, place list and recommendations
Aura	app/(tabs)/aura.tsx	Chat UI, Groq calls, tools
Learn	app/(tabs)/learn.tsx, app/topic/[id].tsx	Lessons, calculator, quiz, topic guides
Wallet	app/(tabs)/wallet.tsx, components/CreditCardImage.tsx	Stacked cards, expiry banner, value summary
Card detail	app/card-detail.tsx	Five tabs of card information
Engine and utilities	utils/recommendationEngine.ts, merchantClassifier.ts, geospatial.ts	Scoring, classification, distance
State and theme	store/useCardStore.ts, theme/	Wallet data, colours, typography

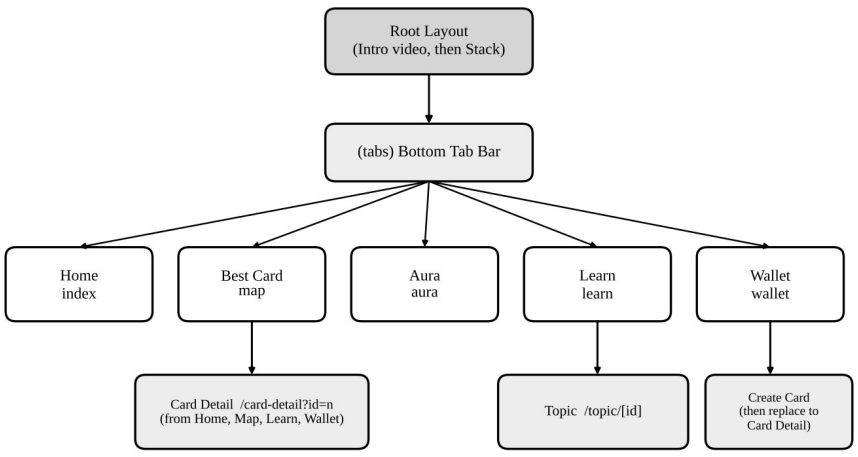


Figure 6.1: Navigation structure of the application

6.2 TOOLS AND TECHNOLOGIES USED

Table 6.2: Technology stack (versions from package.json)

Technology	Version	Purpose
Expo	~57.0.24	Tool chain, native modules; React Compiler and typed routes enabled in app.json
React Native / React	0.86.3 / 19.2.3	Native UI from a JavaScript code base
Expo Router	~57.0.22	File-based navigation, typed routes
TypeScript	~6.0.3	Static typing
Zustand	^5.0.15	Global wallet state
React Native Reanimated	4.5.1	Native-thread animations, shared transitions
react-native-maps	1.27.2	Map view and markers with Mapbox raster tiles
expo-location	~57.0.19	Foreground location permission and updates
expo-video / expo-image	~57.0.4 / ~57.0.5	Intro video; cached card images
AsyncStorage	2.2.0	Installed for planned persistence
Groq API (gpt-oss-120b)	Hosted	Language model for Aura
Mapbox	Geocoding v5, dark-v11 tiles	Place search, nearby places, map style

6.3 ALGORITHM DETAILS

6.3.1 Algorithm 1: Merchant Normalisation and Classification

Mapbox returns raw names such as "Bharat Petroleum Pump" or "Croma - Viman Nagar". `normalizeMerchant` lower-cases the name, resolves known aliases to a canonical brand (for example Bharat Petroleum to BPCL), strips noise suffixes and title-cases the result. `classifyCategory` concatenates the lower-cased name with the Mapbox category and returns the first category whose keyword list contains a matching substring; if none matches it returns Unknown, and the hook then falls back to the category chip that the user selected.

```
function classifyCategory(name, mapboxCategory) {
  const s = `${name.toLowerCase()} ${(mapboxCategory || "").toLowerCase()}`;
  for (const [category, keywords] of Object.entries(CATEGORY_MAP))
    for (const kw of keywords) if (s.includes(kw)) return category;
  return "Unknown";
}
```

6.3.2 Algorithm 2: Rule-Based Card Scoring

`calculateCardScore` implements the model of Section 4.2. It begins with the catalogue term, applies the first matching merchant or category rule to set the bonus and the estimated value, uses the fallback base rate if the value is still zero, and returns the card, the score, the value and the reason string. `getRankedRecommendations` maps this over the wallet and sorts in descending order of score.

6.3.3 Algorithm 3: Nearby Places Pipeline with Stale-Request Guard

When the location or category changes, the hook maps the category to a search keyword (Dining to restaurant), requests up to twelve nearby places, enriches each with its normalised name, category, haversine distance and ranked recommendation, and sorts by distance. Each request is tagged and its result is applied only if it is still the latest, so a late response cannot overwrite a newer one; the autocomplete search does the same with a 400 ms debounce. Location updates use balanced accuracy every five seconds or ten metres, and readings worse than one kilometre are ignored.

6.3.4 Algorithm 4: Aura Tool-Calling Loop

Aura keeps a message list that starts with the system prompt and posts it, with the two tool definitions (`get_user_cards`, `calculate_best_card_for_spend`) and automatic tool choice, to the Groq endpoint at temperature 0.2 and 1,024 maximum tokens. If the reply contains tool calls, the executor runs each against the card store, appends the results and calls the model again; otherwise the reply is shown. Null content is replaced by empty strings before sending (it failed on iOS), and a network or parsing error produces an apology message.

6.3.5 Algorithm 5: Points Calculator and Expiry Alert

The Learn calculator multiplies the spend by an indicative rate for the selected card (3.3% Infinia, 2.5% Octane, 2.0% Platinum Travel, 1.5% otherwise). The wallet's expiry banner selects the first card whose `expiryDays` is 30 or fewer, which in the seed data is BPCL Octane with 4,200 points and 18 days left.

7. SOFTWARE TESTING

7.1 TYPE OF TESTING

Testing combined automated and manual techniques. The merchant classifier and the recommendation engine were run directly, unchanged, against the seed wallet; the whole code base was checked with the TypeScript compiler and ESLint; a production web bundle confirmed that every route compiles; and the screens were rendered in Chromium at a phone-sized viewport. Tests that need a device, a GPS fix or live keys are listed separately as device-level tests.

7.2 TEST CASES & TEST RESULTS

7.2.1 Recommendation Engine

Ten merchant and category pairs were scored for the five seed cards at the default spend of ₹2,000. "Expected" is the card that the design of Section 4.2 intends to win; observed values are the engine's real outputs.

Table 7.1: Recommendation engine test cases (spend = ₹2,000)

ID	Merchant / category	Expected top card	Observed top card	Score (value ₹)	Runner-up	Result
TC-1	BPCL / Fuel	BPCL Octane	BPCL Octane	246.8 (145)	Infinia	Pass
TC-2	Marriott / Travel	Marriott Bonvoy	Marriott Bonvoy	403.5 (300)	Infinia 397	Pass
TC-3	Taj / Travel	Platinum Travel	Infinia	397.0 (320)	Platinum 284.8	Fail (D-1)
TC-4	Starbucks / Dining	Infinia	Infinia	732.0 (660)	Regalia 38.5	Pass
TC-5	Croma / Electronics	Infinia	Infinia	372.0 (320)	Regalia 138.5	Pass
TC-6	DMart / Grocery	Infinia (base)	Infinia	78.0 (66)	Regalia 38.5	Pass*
TC-7	Amazon / Shopping	Infinia	Infinia	372.0 (320)	Regalia 138.5	Pass
TC-8	Apollo Pharmacy / Health	Infinia (base)	Infinia	78.0 (66)	Regalia 38.5	Pass*
TC-9	PVR / Entertainment	Infinia (base)	Infinia	78.0 (66)	Regalia 38.5	Pass*
TC-10	IndiGo / Travel	Infinia	Infinia	397.0 (320)	Regalia 38.5	Pass

*The engine has no grocery, health or entertainment rules, so these cases use the base rate; the result is correct for the implemented rules but shows where the rule set is thin. TC-3

revealed a defect: the Taj rule is attached only to Platinum Travel, while the Travel category rule is evaluated for every other card, so Infinia outscores it at the merchant for which the design favours Platinum Travel. An empty wallet returned an empty list, as required.

7.2.2 Merchant Classifier

Table 7.2: Merchant normalisation and classification test cases

ID	Input name	Normalised name	Category returned	Result
TC-11	Starbucks Coffee	Starbucks	Dining	Pass
TC-12	Croma - Viman Nagar	Croma	Electronics	Pass
TC-13	Bharat Petroleum Pump	BPCL	Unknown (expected Fuel)	Fail (D-2)
TC-14	D-Mart	DMart	Unknown (expected Grocery)	Fail (D-3)
TC-15	McDonald's	McDonalds	Unknown (expected Dining)	Fail (D-4)

7.2.3 Defects Found

Table 7.3: Defect log

ID	Observation	Cause and proposed fix
D-1	Taj / Travel recommends Infinia instead of Platinum Travel	Merchant rule covers one card only. Add the merchant bonus on top of the category rule
D-2	"Bharat Petroleum Pump" is classified Unknown	Fuel keywords lack "petroleum". Classify the normalised name and add keywords
D-3	"D-Mart" is classified Unknown	Keyword "dmart" misses the hyphenated form. Strip punctuation before matching
D-4	"McDonald's" is classified Unknown	Keyword "mcdonalds" misses the apostrophe form. Strip punctuation before matching
D-5	Aura uses its own multipliers (code inspection)	Aura's mock rules test "Gas" while the app uses "Fuel", so it can disagree with the map. Reuse the shared engine

Defects D-2 to D-4 are softened because an Unknown category falls back to the selected category chip. All five defects are scheduled in Section 9.2.

7.2.4 Static Analysis and Build Verification

Table 7.4: Static analysis and build results

Check	Result	Details
tsc --noEmit	3 errors	Web-only style property in index.tsx and two missing CSS declarations in web files; none affects native builds
expo lint (ESLint)	50 problems: 14 errors, 36 warnings	Mainly react-hooks/set-state-in-effect (for example in useBestCardLogic.ts) and unused variables
expo export -p web	Succeeded	Static bundle generated for all routes

7.2.5 UI Smoke and Device-Level Tests

Table 7.5: UI smoke tests (web build, 390 × 844 viewport) and device-level tests

ID	Test	Expected behaviour	Result
UI-1	Open Home, Wallet, Learn	Greeting, snapshot, stacked cards, expiry banner, search and featured video render	Pass
UI-2	Open Aura	Greeting bubble, suggestion chips, input bar render	Pass (UI only)
UI-3	Open Card detail (id = 1) and Topic guide (travel)	Infinia art, 24,000 points and tabs; ranked travel cards render	Pass
UI-4	Open Best Card on web	Not supported: react-native-maps is native-only	Fails on web (expected)
DV-1	Map with GPS on device; permission denied	Pins, sheet and recommendations appear; on denial loading ends and manual search remains	To verify on device
DV-2	Place search autocomplete	Results after 400 ms; selecting re-centres the map	To verify on device
DV-3	Aura tool call with live key	Tool is executed and a grounded answer is shown	To verify on device

8. RESULTS

8.1 OUTCOMES

The project delivered a working, polished prototype that covers the full loop from learning, through choosing a card, to understanding redemption. Table 8.1 maps each objective of Section 1.3.2 to its outcome.

Table 8.1: Objectives versus outcomes

Objective	Outcome	Status
Card wallet	Five cards with fees, benefits, milestones, lounge access, forex, points and value range	Achieved (read-only)
Best card for a place	Map with nearby places, classification and a ranked recommendation	Achieved (rule-based)
Rupee value of a swipe	Estimated value and reason shown with each recommendation	Achieved
AI assistant	Aura with two tools, domain restriction and error handling	Achieved (live test pending)
Learn hub	Topics, calculator, quiz, myth versus fact, academy, travel guide	Achieved (one topic populated)
Expiry warning	Banner for points expiring within 30 days	Achieved
Premium interface	Dark gold theme, shared transitions, gesture-driven sheet	Achieved

8.2 SCREEN SHOTS

Figure 8.1 is a schematic of the Best Card screen, which depends on native map modules; Figures 8.2 to 8.4 are grey-scale screenshots of the other screens captured from the web build at 390 × 844. Home shows the greeting, financial snapshot, Aura card and the rewards optimiser (Dining, ₹5,000, best card HDFC Infinia). Wallet shows the stacked deck, portfolio value and the expiry banner for BPCL Octane (4,200 points, 18 days left).

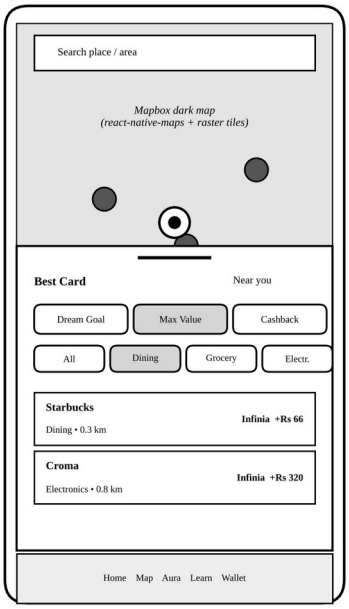


Figure 8.1: Schematic of the Best Card (map) screen

Drawn from the implemented layout because the map module runs only on devices.

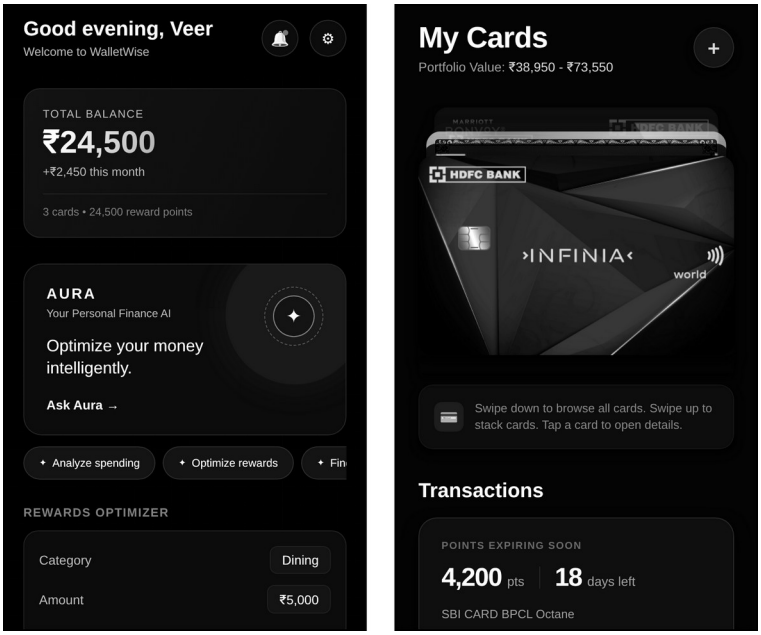


Figure 8.2: Home and Wallet screens

(a) Home (b) Wallet

Aura greets the user and offers three suggestion chips; Learn offers search, the next lesson with a progress bar and the featured video "How Credit Card Rewards Work".

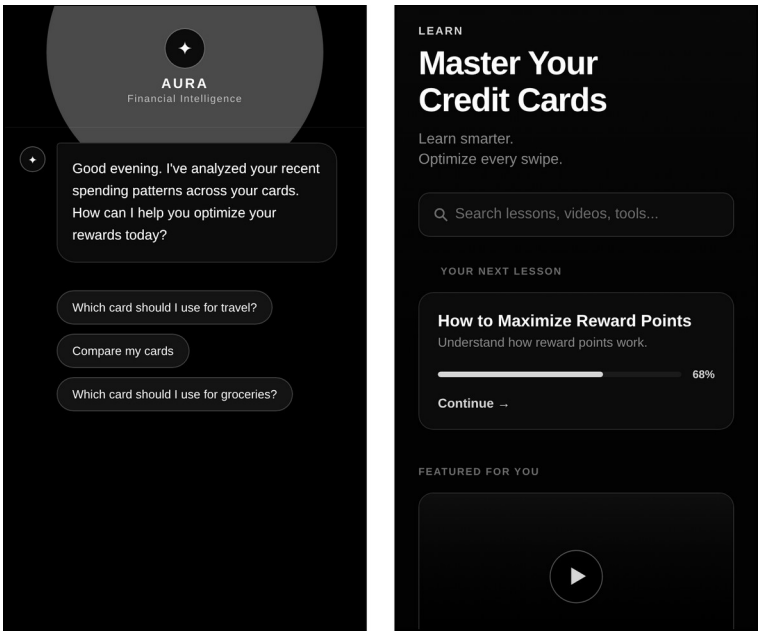


Figure 8.3: Aura and Learn screens
(a) Aura (b) Learn

Card Detail shows the Infinia artwork, 24,000 points worth ₹12,000 to ₹24,000, and the tab bar (Earn: 3.3% base return, five points per ₹150). The Topic screen ranks HDFC Infinia Metal first among the "Top 10 Travel Cards in India" and tabulates tier, fees, forex, reward range and fee-waiver limit with an "Apply Now" link.

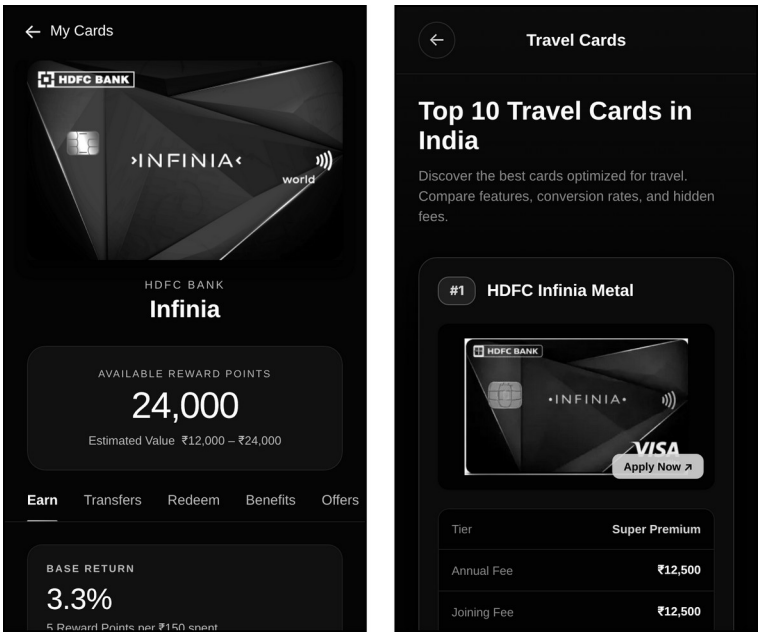


Figure 8.4: Card Detail and Topic guide screens
(a) Card Detail (Infinia) (b) Topic guide (Travel)

9. CONCLUSIONS

9.1 CONCLUSIONS

WalletWise demonstrates that the knowledge that experienced cardholders use to turn everyday spending into free travel can be packaged for a beginner in a single mobile application. The project met its main objectives: it stores a card wallet, recommends a card for nearby merchants through a transparent rule-based engine that also states the reason and the rupee value, offers a domain-restricted AI assistant that consults the wallet through tools, and teaches the basics through a Learn hub, all in a consistent premium interface built from one TypeScript code base.

The evaluation also shows where the prototype must grow. Testing exposed one scoring defect and three classification gaps, the static analysis reports a small number of type and lint errors, card data and rules are still hard-coded, and keys are bundled with the client. These findings are expected in a first iteration, and they indicate the work that will turn the prototype into a dependable product.

9.2 FUTURE WORK

1. **Dataset-driven catalogue.** Replace the hard-coded cards and rules with an imported, versioned dataset such as the CardAdvisor open data (279 cards, CC BY 4.0), storing reward rates, caps, fees, lounge benefits and point values in a structured schema.
2. **Single shared engine.** Make the map, the calculator and Aura use the same engine, remove the duplicate mock rules, fix defects D-1 to D-5, and extend the rules to groceries, health and entertainment.
3. **Point valuation and redemption comparison.** Value every point in rupees for each redemption route (statement credit, vouchers, airline and hotel transfers) and rank routes, in the manner of PointsFly's valuation, with a rule of thumb for good and poor value.
4. **Voucher redemption guidance.** Show which voucher platforms accept a card's points and the effective discount, so that points can be used for dining, groceries, healthcare and entertainment.

5. **Dream-destination tracker.** Let the user choose a trip, convert its cost into points, and show progress and the best spending plan to reach it.
6. **Portfolio optimisation.** Model monthly caps, milestones and annual-fee break-even to split spending across cards; this is the optimisation problem analysed in Appendix A.
7. **User-managed wallet and persistence.** Add, edit and delete cards, store them with AsyncStorage, and later synchronise through an authenticated back end.
8. **Back-end proxy and security.** Hold the Groq and Mapbox keys on a server, add rate limiting, and move Aura's data access behind an API.
9. **Knowledge feed and quality.** Add a section that publishes new knowledge (devaluation alerts, offers, tutorials), resolve the type and lint findings, and add automated unit, end-to-end and device tests.

9.3 APPLICATIONS

- **Personal finance education** for students and first-time cardholders in colleges and workplaces.
- **Everyday card selection** at restaurants, fuel stations, supermarkets and electronics stores.
- **Travel planning**, in which accumulated points are targeted at flights and hotel stays.
- **Financial-literacy products** for banks and fintech companies as a white-label learning module.
- **Research and teaching** as a case study of rule-based recommendation, location services and tool-calling language models in mobile applications.

APPENDIX A: FEASIBILITY ASSESSMENT

This appendix analyses the computational feasibility of the two decision problems that WalletWise addresses, using the notation of Section 4.2 and the complexity classes of Garey and Johnson [24].

A.1 Problem 1: Best Card for a Single Purchase

Given a finite set of cards C, a merchant m, a category k and an amount a, find c* = argmax S(c, m, k, a). Each score is computed in constant time from a fixed rule table, so the whole selection requires n evaluations and a maximum, that is O(n) time. The decision version, "is there a card with score at least t?", is therefore decidable in polynomial time, and the problem belongs to class **P**. No satisfiability reasoning is needed because every rule is an independent boolean test on the merchant and category.

A.2 Problem 2: Allocating a Monthly Budget Across Cards

The planned portfolio optimisation assigns, for each spending category j with budget b_j, a part x_{ij} of the budget to card i so as to maximise total reward value, subject to per-card monthly caps and spend milestones that unlock a bonus (for example ₹15,000 points at ₹1.9 lakh). A milestone introduces a binary decision $y_i = 1$ if the spend on card i reaches its threshold T_i, which gives the mixed-integer model

$$\text{maximise } \sum r_{ij} \cdot x_{ij} + \sum g_i \cdot y_i \quad \text{subject to } \sum_i x_{ij} = b_j, \quad \sum_j x_{ij} \geq T_i \cdot y_i,$$
$$\sum_j x_{ij} \leq \text{cap}_i, \quad y_i \in \{0, 1\}$$

With a single category and no caps the problem degenerates to a greedy choice. With thresholds it contains the *0-1 knapsack problem* as a special case: items correspond to cards, the weight of an item is its threshold spend T_i, the profit is its bonus g_i, and the knapsack capacity is the total budget. Knapsack is NP-complete in its decision form [24], so the general portfolio problem is **NP-hard**. It is, however, only *weakly* NP-hard: the budget is an integer of modest size (in rupees or thousands of rupees), so a dynamic-programming algorithm solves it in pseudo-polynomial time $O(n \cdot B)$, and a wallet has only a handful of cards. The problem is thus practical to solve exactly on a device, and the roadmap item of Section 9.2 is feasible.

APPENDIX B: DETAILS OF PAPER PUBLICATION

No research paper based on this project has been submitted to a conference or journal at the time of writing, so there are no reviewer comments or publication certificates to attach. This page is reserved for the name of the conference or journal, the reviewers' comments, the certificate and the paper, should the work be published later.

APPENDIX C: PLAGIARISM REPORT

The plagiarism check report generated for this project report by the institute-approved software is to be attached on this page before submission, as required by the report guidelines.

REFERENCES

[1] CardAdvisor, "Credit card recommender and rewards guide for India." [Online]. Available: https://cardadvisor.in/

[2] CardAdvisor, "How credit card reward points work in India." [Online]. Available: https://cardadvisor.in/articles/how-credit-card-reward-points-work-india

[3] PointsFly, "Company profile," YourStory. [Online]. Available: https://yourstory.com/companies/pointsfly

[4] GyFTR, "Get up to 20% off on gift vouchers using IndusInd credit cards." [Online]. Available: https://www.gyftr.com/blog/get-up-to-20-percent-off-on-gift-vouchers-using-indusind-credit-cards/

[5] PointsFly, "Tool listing and pricing," Toolradar. [Online]. Available: https://toolradar.com/tools/pointsfly

[6] CardAdvisor, "Open credit card dataset (CC BY 4.0)." [Online]. Available: https://cardadvisor.in/data

[7] Axis Bank, "GyFTR EDGE Rewards terms and conditions." [Online]. Available: https://www.axisbank.com/docs/default-source/default-document-library/gyftr-tnc-26sep.pdf

[8] Mapbox, "Geocoding API (v5) documentation." [Online]. Available: https://docs.mapbox.com/api/search/geocoding-v5/

[9] R. W. Sinnott, "Virtues of the haversine," Sky and Telescope, vol. 68, no. 2, p. 159, 1984.

[10] Expo, "Location (expo-location) SDK reference." [Online]. Available: https://docs.expo.dev/versions/latest/sdk/location/

[11] Groq, "Groq API documentation: chat completions and tool use." [Online]. Available: https://console.groq.com/docs/overview

[12] OpenAI, "gpt-oss-120b and gpt-oss-20b model card," arXiv:2508.10925, 2025.

[13] Meta Platforms, "React Native documentation." [Online]. Available: https://reactnative.dev/docs/getting-started

[14] Expo, "Expo documentation." [Online]. Available: https://docs.expo.dev/

[15] Expo, "Introduction to Expo Router." [Online]. Available: https://docs.expo.dev/router/introduction/

[16] Poimandres, "Zustand: a small, fast and scalable state-management solution." [Online]. Available: https://github.com/pmndrs/zustand

[17] Software Mansion, "React Native Reanimated documentation." [Online]. Available: https://docs.swmansion.com/react-native-reanimated/

[18] react-native-maps contributors, "react-native-maps." [Online]. Available: https://github.com/react-native-maps/react-native-maps

[19] ISO/IEC/IEEE 29148:2018, Systems and Software Engineering: Life Cycle Processes: Requirements Engineering, 2018.

[20] I. Sommerville, Software Engineering, 10th ed. Harlow, U.K.: Pearson, 2016.

[21] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 8th ed. New York, NY, USA: McGraw-Hill, 2014.

[22] G. Booch, J. Rumbaugh and I. Jacobson, The Unified Modeling Language User Guide, 2nd ed. Boston, MA, USA: Addison-Wesley, 2005.

[23] B. W. Boehm, Software Engineering Economics. Englewood Cliffs, NJ, USA: Prentice Hall, 1981.

[24] M. R. Garey and D. S. Johnson, Computers and Intractability: A Guide to the Theory of NP-Completeness. San Francisco, CA, USA: W. H. Freeman, 1979.